

Paginação e Memória Virtual — Visão Conceitual

Páginas (virtuais) ↔ Quadros/Frames (físicos) com mapeamento arbitrário via tabela de páginas

MEMÓRIA VIRTUAL DO PROCESSO (contígua, lógica)

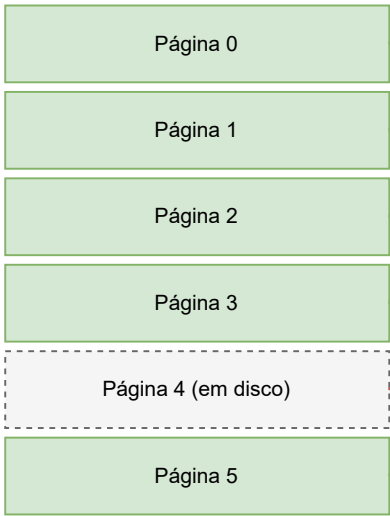
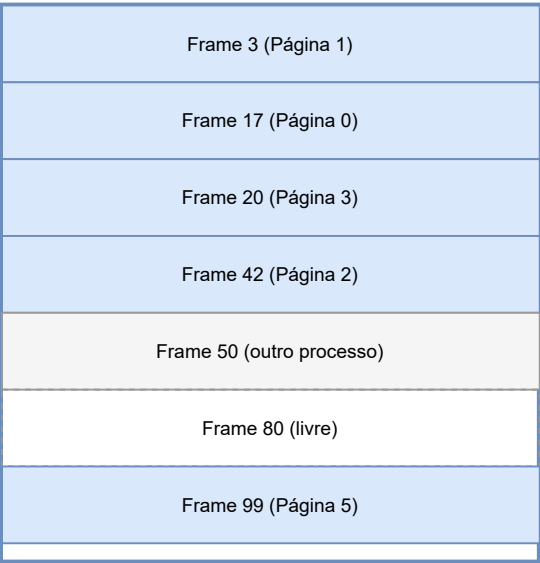


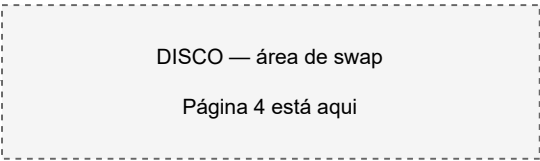
TABELA DE PÁGINAS (hierárquica, mantida pelo kernel)

Página Virt.	Frame	P	R/W	U/S	A	D	NX
0	17	1	0	1	1	0	0 (-text)
1	3	1	1	1	1	1	1 (data)
2	42	1	1	1	0	0	1
3	20	1	1	1	1	1	1
4	(disco)	0	--- página em swap ---				
5	99	1	1	1	0	0	1

MEMÓRIA FÍSICA (RAM) (frames espalhados)



CR3 (registrador da CPU)
aponta para a raiz da tabela
(trocado em cada context switch)



Acesso à Página 4 → PAGE FAULT

- MMU verifica PTE: P=0 → exceção. Kernel detecta tipo:
- major fault: traz a página da área de swap para um frame livre
 - atualiza PTE (P=1, frame correto)
 - invalida TLB
 - retoma instrução (que agora terá sucesso)

TABELA HIERÁRQUICA (x86-64, 4 níveis)

Endereço virtual de 48 bits dividido em 4 índices de 9 bits + offset de 12 bits.

CR3 → PGD → PUD → PMD → PTE → Frame + Offset

Cada nível: 512 entradas (8 bytes × 512 = 4 KiB = uma página).
Tabelas vazias NÃO são criadas — economiza muito espaço.
CPUs novas têm 5 níveis (PML5) cobrindo 128 PiB.

SUBSTITUIÇÃO DE PÁGINA (quando RAM enche)

FIFO
mais antiga sai
ruim

LRU
menos recentemente usada
ótimo • impl. cara

Clock / LRU aprox
bit A + ponteiro circular
usado na prática

Linux: 2 listas LRU
active + inactive
kswapd faz reclaim

Working Set
páginas usadas no intervalo
teórico

LFU
menos frequente
sofre mudança de fase

THRASHING — o ponto de colapso

Quando o working set total dos processos excede a RAM, o kernel:
1) manda página A para swap | 2) processo precisa de A → traz de volta | 3) para trazer, manda B para swap | 4) processo precisa de B → traz...
Resultado: sistema passa mais tempo paginando do que executando. CPU baixa, disco 100%, processos congelados. **Solução:** matar processos (OOM killer) ou adicionar RAM.